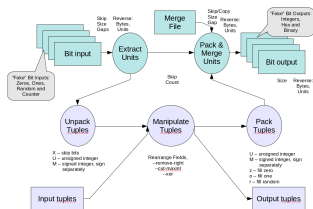


bdd — Bit Data Dump & Stream Engine

The Swiss Army knife for bitstreams. Slice, transform, arithmetic-scale, bitwise-mask, fuse, project, and transcode unaligned binary streams of arbitrary bit widths at multi-gigabit speeds.

Originally created by Esa Turtiainen in Python 2 (2010), 2026 `bdd` was re-engineered in modern safe Rust for memory safety, zero external C dependencies, and line-rate multi-gigabit throughput.

Available as **Fedora / RHEL RPM**, **Debian / Ubuntu DEB**, **Rust library crate**, **C SDK**, **Python SDK**, and native **MCP Server**.



1. Why bdd? The Byte-Boundary Trap vs. Single-Bit Reality

The Problem: Standard Tools are Trapped at Byte Boundaries

Most Unix tools (`dd` , `hexdump` , `od` , standard shell pipes) operate strictly on byte (8-bit) or machine-word (16, 32, 64-bit) boundaries. When you run `dd bs=1` , you are still constrained to 8-bit multiples.

The Reality: Real-World Data Lives at Arbitrary Bit Widths

Real-world binary data does not align neatly to 8-bit bytes:

- **IoT & Embedded Telemetry:** 12-bit ADC raw samples, 10-bit DAC calibration values, 1-bit status flags.
- **Broadcast Multimedia:** 13-bit PIDs in MPEG-TS 188-byte containers, unaligned 32-bit MP3 frame headers, 4-bit JPEG chroma nibbles.
- **Modern AI Inference:** NVIDIA Blackwell 4-bit floats (NVFP4 E2M1), OCP Microscaling 6-bit floats (FP6 E3M2), OCP 8-bit floats (FP8 E4M3/E5M2), Brain Float 16 (BF16).
- **Network Headers:** IPv4 4-bit IHL, TCP 1-bit control flags (`SYN` , `ACK` , `FIN`), unaligned DSCP/ECN QoS fields.
- **Kernel Internals:** Linux Netlink connector sockets, `/proc/[pid]/pagemap` 64-bit page table entries with 55-bit PFNs and discrete single-bit flags.

Attempting to slice, manipulate, or transcode these unaligned fields with standard shell scripts or ad-hoc C programs requires dozens of lines of brittle manual bitshifts, bitwise masks, and fragile endianness adjustments.

The Solution: Declarative Pattern Stream Transformation

`bdd` brings sub-byte surgical precision to the Unix pipeline. Instead of writing throwaway C structs or slow Python glue scripts full of bitshifts and bitmasks, you describe your transformations directly using intuitive **pattern arrow expressions**:

```
# Repack 8-bit bytes into a dense stream of 3-bit units:
echo -ne "\x00\x07\x00\x07\x00\x07" | bdd "8 → 3 → bits"
# Output: 000 111 000 111 000 111

# File-to-file bitstream transcoding (pack 8-bit bytes into non-byte-aligned 3-bit units):
bdd "8 → 3" < foo.8bit > foo.3bit

# Synthesize arbitrary sub-byte bitstreams from human-readable numbers:
printf "%s\n" 0 7 0 7 0 7 | bdd "tuples → 8 → 3 → bits"
# Output: 000 111 000 111 000 111

# Pack comma-separated decimal tuples directly into 3-bit fields:
echo "0,7,0,7,0,7" | bdd "tuples → 6*3u → bits"
# Output: 000111000111000111

# Slices two 4-bit nibbles, swaps their positions, and emits formatted hex:
echo -ne "\xfa" | bdd "4U4U → {1, 0} → 4U4U → hex"
# Output: af

# Invert bits 4..12 of a 16-bit word in place; preserve surrounding bits "as is":
echo -ne "\x12\x34" | bdd "16[4:8] → xor(0xFF) → overwrite → hex"
# Output: 1dc4

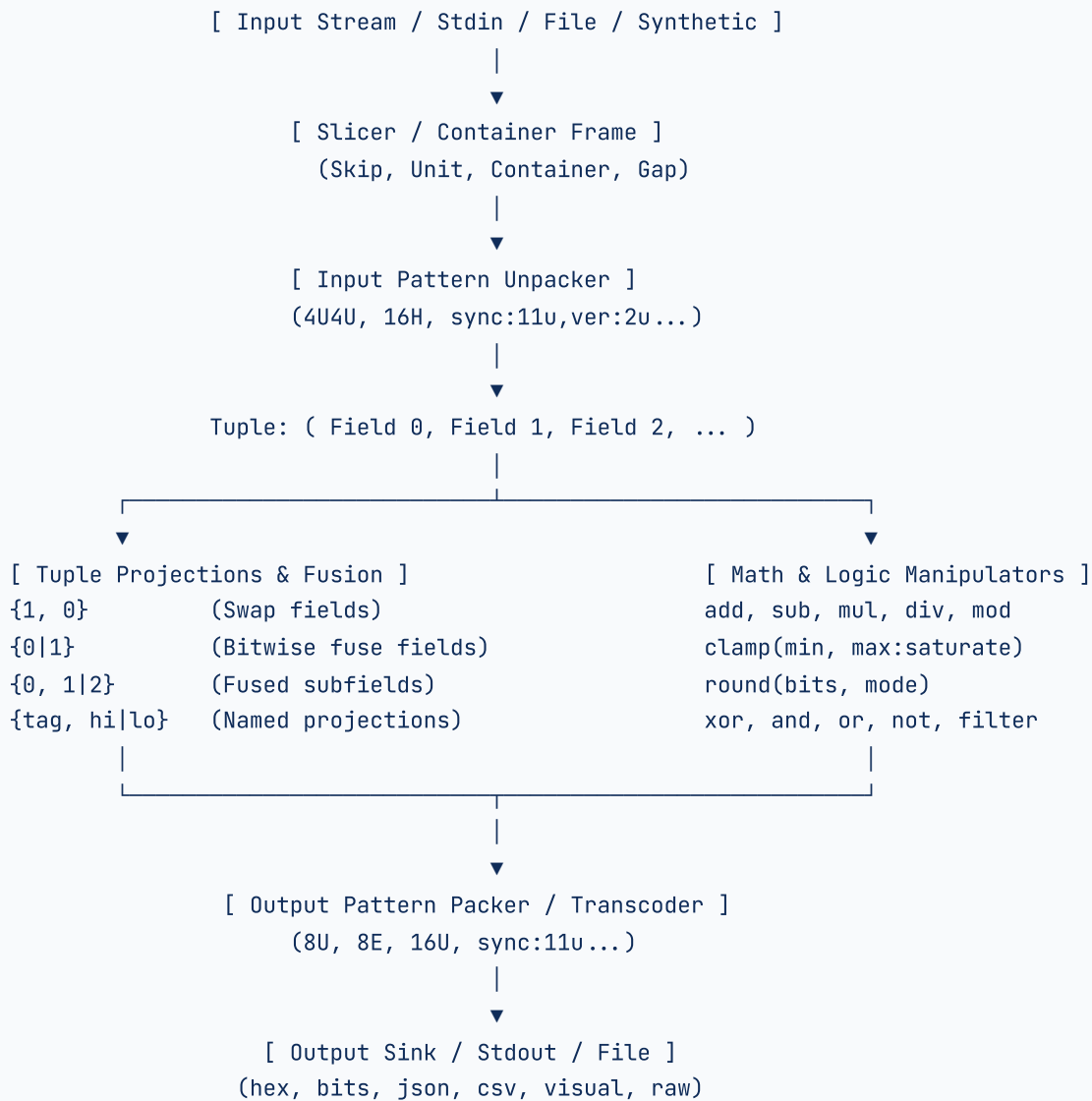
# Interleave two independent streams round-robin:
bdd "[ zeros:8, ones:8 ] → hex" --count 2
# Output: 00 ff 00 ff

# Preserve unparsed binary blobs with wildcard passthrough (_):
echo -ne "\x12\x34" | bdd "4_, 8U, 4_ → xor(1, 0xFF) → 4_, 8U, 4_ → hex"
# Output: 1dc4

# Quantize FP16 neural network weights directly to NVIDIA Hopper OCP FP8 (E4M3):
bdd "16H → 8E" < model_fp16.bin > model_fp8.bin

# Calibrate 12-bit ADC sensor readings, remove DC bias, amplify, and clamp to 8-bit DAC:
bdd "12 → sub(512) → mul(2) → clamp(0, 255:saturate) → 8" < adc.raw > dac.raw

# Extract 13-bit MPEG-TS PIDs from repeating 188-byte container frames:
bdd "188B[11:13] → 13" < stream.ts
```



2. The Core Paradigm: Unified Stream Arrow Pipelines

The fundamental interface of `bdd` is the **Stream Arrow Transformation Pipeline**:

```
[source → ] [container / slicer → ] [pattern → ] [manipulator...] [ → pattern] [ → slicer] [ → sink]
```

Instead of passing dozens of disconnected CLI flags, you describe the entire lifecycle of your bitstream in a single readable pipeline string:

```
bdd "stdin → 8 → 4U4U → {0|1} → 16U → 16 → stdout"
```

Zero Redundant Dimensions: Automatic Bit-Width Derivation

In `bdd`, **pattern declarations automatically define input and output unit bit sizes**.

You do **not** need to specify redundant `--input-unit` or `--output-unit` flags:

- Declaring `16H` automatically sets the input unit size to **16 bits**.
- Declaring `4U4U` automatically sets the unit size to **8 bits** (4 + 4).
- Declaring `8E` automatically sets the output unit size to **8 bits**.
- Declaring `sync:11u,ver:2u,layer:2u` automatically sets the unit size to **15 bits**.

Therefore, the explicit pipeline above simplifies cleanly to:

```
bdd "stdin → 4U4U → {0|1} → 16U → hex"
```

Or when using default standard input and output:

```
bdd "4U4U → {0|1} → 16U → hex"
```

Built-in Stream Sources

Sources specify where raw bits originate:

SOURCE	DESCRIPTION	EXAMPLE
<code>stdin</code>	Standard input pipe or redirected file (default)	<code>bdd "stdin → 8 → hex"</code>
<code>zeros</code>	Infinite stream of continuous zero bits (<code>00000000...</code>)	<code>bdd "zeros → 8 → hex" --count 4</code>
<code>ones</code>	Infinite stream of continuous one bits (<code>11111111...</code>)	<code>bdd "ones → 8 → hex" --count 4</code>
<code>rand</code>	High-entropy random bits from <code>/dev/urandom</code>	<code>bdd "rand → 16 → hex" --count 2</code>
<code>counter</code>	Monotonically increasing sequential numbers (<code>0,1,2,3...</code>)	<code>bdd "counter → 8 → hex" --count 4</code>
<code>file(path)</code>	Direct file read without shell redirection	<code>bdd "file('dump.bin') → 8 → hex"</code>
<code>tuples</code>	Reads comma-separated text tuple lines or newline-separated integers from standard input	<code>printf "%s\n" 0 7 0 7 bdd "tuples → 8 → 3 → bits"</code>

Built-in Stream Sinks

Sinks specify how the processed bits or tuples are formatted on output:

SINK	DESCRIPTION	EXAMPLE OUTPUT
<code>stdout</code>	Raw binary bitstream written to standard output (default)	Binary bytes
<code>hex</code>	Space-delimited lowercase hexadecimal byte/word strings	<code>00 fa</code>
<code>bits</code>	Clean ASCII bit strings (<code>0</code> and <code>1</code>)	<code>11111010</code>
<code>json</code>	Newline-delimited JSON arrays (NDJSON)	<code>[15, 10]</code>
<code>json:object</code>	Keyed JSON dictionaries using pattern field names	<code>{"sync":71,"pid":256}</code>
<code>csv</code>	Standard comma-separated values	<code>15,10</code>
<code>visual</code>	Interactive ANSI colorized terminal dump with bitfield borders	Color matrix
<code>integers</code>	Unsigned base-10 integer per unit (one per line)	<code>250</code>
<code>file(path)</code>	Direct file write without shell redirection	<code>bdd "... → file('out.bin')</code>

Containers and Units: Framing the Bitstream

Active data rarely floats in empty space; interesting bits reside inside physical framing containers (such as 188-byte MPEG-TS packets, 32-bit registers, or padded network frames).

- **Container:** A repeating bit unit in the stream that can be defined with an initial `skip` and periodic `gap` .
- **Unit:** The interesting bits inside the container that can be found using `length` and `pregap` (offset from container start), or symmetrically using `pregap` , `length` , and `postgap` .

Rather than forcing you to mentally calculate offsets and trailing padding, `bdd` provides declarative container notation:

0. Pure Bit-Width Slicing: `in_unit → out_unit` (e.g. `8 → 3`)

Directly reslices and repacks continuous bitstreams between arbitrary bit widths without manual bitmasks or shifts:

- **8 → 3** : Slices 8-bit input units into dense 3-bit output units (e.g. packing byte values into 3-bit octal fields).
- **3 → 8** : Expands 3-bit units into byte-aligned 8-bit units.

```
bdd "counter → 8 → 3 → hex" --count 8
# Output: 0 1 2 3 4 5 6 7
```

1. Form A Containers: `container[pregap:length] (or raw[offset:unit] )`

Extracts the interesting **unit** of `length` bits starting at `pregap` (offset) inside a repeating outer **container**:

- **8[2:4]** : Inside each 8-bit container, extract the 4-bit unit starting at bit offset 2 (skipping 2 bits before, and 2 bits after).
- **188B[11:13]** : Inside each 188-byte container, extract the 13-bit PID unit starting at bit offset 11 (automatically skipping the remaining 1,480 bits of container payload).
- **32[0:16]** : Extract the 16-bit Left channel unit from a 32-bit container.
- **32[16:16]** : Extract the 16-bit Right channel unit from a 32-bit container.

```
bdd "counter → 8[2:4] → hex" --count 4
# Output: 0 1 2 3
```

2. Form B Containers: `[pregap:length:postgap]`

Expresses symmetrical container framing by specifying pregap, unit length, and postgap:

- **[2:4:2]** : 2 bits pregap, 4 bits unit length, 2 bits postgap (repeating container of 8 bits: $2 + 4 + 2 = 8$).

```
bdd "counter → [2:4:2] → hex" --count 4
# Output: 0 1 2 3
```

3. Container Skips & Periodic Gaps

- **Initial Skip (`skip : container`)**: Number of bits to skip before the first container is read. **16 : 8** skips 16 bits (2 bytes) into the stream and begins streaming 8-bit containers.
- **Periodic Gap (`container + gap`)**: Number of bits to skip after every container. **8 + 24** reads an 8-bit container, skips a 24-bit gap, and repeats (extracting 1 byte out of every 4-byte container frame).
- **Full Unified Syntax**: **123 : 188B[11:13] + 8 → 13** (Initial skip of 123 bits, 188-byte container with a 13-bit unit at pregap 11, periodic inter-container gap of 8 bits, emitted as 13-bit units).

```
# Read an 8-bit container, then skip a 24-bit periodic gap (1 byte out of every 4 bytes):
bdd "counter → 8 + 24 → hex" --count 4
# Output: 00 01 02 03

# Skip 16 bits (2 bytes) initially, then extract 8-bit units:
bdd "counter → 16 : 8 → hex" --count 4
# Output: 00 01 02 03
```

In-Place Container Editing with `overwrite` : Preserving Skips, Gaps, and Headers

When slicing structured containers like `188B[11:13]`, standard extraction isolates the 13-bit payload and discards the surrounding 1,491 bits (the 11-bit prefix and 1,480-bit suffix). If you repack the modified stream back into `188B[11:13]` using a standard output slicer, those unselected bits are zeroed out by default.

In reverse-engineering, firmware patching, and broadcast engineering, you often need to **modify a specific field in place while preserving everything else untouched**—including pre-offset headers, post-offset payloads, initial skips, and periodic framing gaps.

Add `overwrite` (or `overwrite(...)`) to your pipeline:

```
# Invert bits 4..12 of a 16-bit word in place; preserve bits 0..4 and 12..16 "as is":
bdd "16[4:8] → xor(0xFF) → overwrite → hex"

# In-place PID rewriting in a broadcast MPEG-TS transport stream:
# Reads 188-byte packets, XOR-patches the 13-bit PID at bit offset 11,
# and writes back the entire 188-byte packet with sync bytes, flags, and payloads intact:
bdd "file('broadcast.ts') → 188B[11:13] → xor(0x1FFF) → overwrite → file('patched.ts')"
```

Under `overwrite` mode:

1. **Container Integrity:** The base stream reads full containers directly. Only the sliced bit window is extracted and piped through the transformation stages.
2. **Bit-Level Splicing:** The manipulated value is spliced back into the original container bits at the exact bit offset using bitmasking, leaving all surrounding header, payload, and checksum bits identical to the input.
3. **Preserved Skips & Gaps:** Any initial stream skip bits (`skip : ...`) or periodic framing gaps (`... + gap`) are read from the original stream and written directly to the output without zeroing out or truncation.

Multi-Source Stream Coordination: Bracket Syntax [`source1, source2, ...`]

`bdd` allows interleaving multiple independent streams round-robin using bracket syntax at the start of a pipeline:

```
# Interleave 8-bit zeroes and 8-bit ones round-robin (count takes 2 units from both sources):
bdd "[ zeros:8, ones:8 ] → hex" --count 2
# Output: 00 ff 00 ff

# Interleave standard input with 32-bit addresses read from a binary file:
bdd "[ stdin:8, file('addresses.bin'):32 ] → stdout"
```

Identical Stream Syntax Everywhere

Every stream specifier inside brackets shares the **exact same universal stream grammar** as the primary stream:

Source [: Skip] : ContainerOrUnit [+ Gap] [: Modifiers]

Secondary streams can have their own skips, containers, offsets, and gaps:

```
bdd "[ file('stream_a.bin'):188B[11:13], file('stream_b.bin'):1024:16[0:8]+8 ] → hex"
```

Inherited Unit Sizing

If individual streams inside brackets omit the unit width, they automatically inherit the downstream slicer width:

```
# Both 'zeros' and 'counter' streams inherit the 16-bit unit size from the downstream slicer:
bdd "[ zeros, counter ] → 16 → hex" --count 2
# Output: 0000 0000 0000 0001
```

In-Pipeline Stream Interleaving & Padding: `interleave(...)` / `merge(...)`

You can also introduce secondary streams midway through a pipeline using the `interleave(...)` or `merge(...)` pipe operators:

```
# Interleave a primary 8-bit stream with an 8-bit counter:
bdd "zeros:8 → interleave(counter:8) → hex" --count 2
# Output: 00 00 00 01

# Insert synthetic zero units between primary units:
bdd "stdin:16 → interleave(zeros:8) → stdout"
```

Unequal Stream Lengths and `pad:zeros`

When merging physical files of unequal size, the secondary stream may exhaust before the primary stream. By default, reaching EOF on any merge stream terminates round-robin interleaving. To keep the primary

pipeline running and pad the missing secondary units with zeroes, add `pad:zeros` :

```
# Continue streaming primary payload even if tags.bin runs out of bytes:
bdd "stdin:16 → interleave(file('tags.bin'):8, pad:zeros) → stdout"
```

Non-Destructive In-Pipeline Stream Tapping: `tee(...)`

Like the classic Unix `tee` utility, `bdd` supports non-destructive intermediate stream tapping directly inside the arrow pipeline using `tee('filepath')` (or `tee(file('filepath'))`):

```
# Tap intermediate 8-bit units after adding 5, then continue to add 1 and output to hex:
bdd "zeros:8 → add(5) → tee('stage1_tap.bin') → add(1) → hex" --count 2
# stdout: 06 06
# stage1_tap.bin: 05 05
```

`tee(...)` captures the exact bitstream units passing through that point in the transformation graph, writing them to disk without mutating or impeding downstream pipeline stages.

3. Pattern Grammar & Tuples: Slicing Structured Fields

When data contains multiple heterogeneous fields, a single unit size is not enough. You unpack the unit into an ordered **Tuple of Fields** using pattern grammar:

```
bdd "sync:11u,version:2u,layer:2u,protect:1b → json:object" < audio.mp3
```

What is a Tuple in `bdd` ? Containers, Units, Tuples, and Lines

To use `bdd` effectively, understand the relationship between **streams**, **containers**, **units**, and **tuples**:

1. **Bitstream**: The continuous sequence of bits (from file, stdin, network socket, or memory).
2. **Container**: The repeating bit unit in the stream, defined by an initial `skip` and periodic `gap` (e.g. 188-byte MPEG-TS packets, 32-bit audio frames).
3. **Unit**: The interesting bits inside the container, located using `length` and `pregap` (offset from container start), or symmetrically using `pregap`, `length`, and `postgap`. When no container framing is specified, the unit occupies the entire container.
4. **Tuple**: The structured, in-memory breakdown of **one single unit** into typed fields:
 - A unit is a sequence of bits (e.g. 8-bit `0xFA` / `0b11111010`).
 - Slicing it with pattern `4U4U` yields **one tuple** with 2 fields: `[UInt(15), UInt(10)]` .
5. **Text / CSV Representation** (`--input-tuples` / `--output-tuples` / `tuples → ...`):
 - **One Line = Exactly One Unit (One Tuple)**.

- **Commas (,)** separate **fields** within that unit.
- **A newline (\n)** terminates the **unit** and advances the stream to the next unit.

Example: Line vs. Comma Semantics

- **Single line with commas (1 unit with 6 fields):**

```
echo "0,7,0,7,0,7" | bdd "tuples → 6*3u → bits"  
# Output: 000111000111000111 (ONE unit composed of six 3-bit fields)
```

- **Multiple lines with newlines (6 sequential units):**

```
printf "%s\n" 0 7 0 7 0 7 | bdd "tuples → 8 → 3 → bits"  
# Output: 000 111 000 111 000 111 (SIX sequential units, each on its own line)
```

Supported Pattern Specifiers

CODE	TYPE	DESCRIPTION
U	Unsigned Integer (Big-Endian)	Most Significant Bit first across the field width.
u	Unsigned Integer (Little-Endian)	Bit-reversed within the field (LSB first).
S	Signed Integer (Two's Complement, Big-Endian)	Preserves sign bit at MSB.
s	Signed Integer (Two's Complement, Little-Endian)	Bit-reversed within the field.
M	Sign-Magnitude Integer (Big-Endian)	High bit is sign (0 = +, 1 = -), remaining bits are magnitude.
m	Sign-Magnitude Integer (Little-Endian)	Bit-reversed sign-magnitude integer.
b	Boolean Flag	1-bit boolean flag (evaluates to integer 0 or 1).
c / c	Raw Byte Character / Bytes	Preserves raw ASCII or byte characters without lossy UTF-8 conversion.
_	Wildcard Opaque Passthrough Bits	Passes raw bits through without parsing, numerical decoding, or type coercion.
x	Discard / Padding Bits	Consumed from input stream and discarded (never emitted into tuple).
k	Constant Field	Verifies constant magic numbers or synchronization headers.

Opaque Wildcard Passthrough (**_**): Zero-Cost Bitfield Preservation

When working with complex binary structures, you often want to extract or modify only a specific field while leaving all surrounding fields completely untouched:

```
# Invert only the middle 8-bit unsigned byte of a 16-bit word:  
# The 4 leading bits (Field 0) and 4 trailing bits (Field 2) are preserved untouched:  
bdd "4_, 8U, 4_ → xor(1, 0xFF) → 4_, 8U, 4_ → hex"
```

Wildcard notation supports multiple formats:

- **Prefix:** **4_** (4 raw bits) or **184B_** (184 raw bytes)
- **Suffix:** **_4** or **_11**

- **Colon:** `_:4` or `_:13`

Using `_` treats the field as an opaque raw bit sequence (`Field::Bits`). This avoids unnecessary numerical decoding overhead, eliminates sign-extension or rounding risks, and guarantees bit-perfect serialization when packing the tuple back into an output bitstream.

Floating-Point Codecs & The Universal `f64` Currency

`bdd` includes zero-dependency pure-Rust codecs for modern AI, GPU, and DSP floats:

FLOAT CODEC	TOTAL BITS	EXPONENT / MANTISSA	STANDARD & USAGE
<code>16F</code>	16	5 exp / 10 man	IEEE 754 Half-Precision Float (binary16).
<code>16H</code>	16	5 exp / 10 man	IEEE 754 Half-Precision Float (alias for <code>16F</code>).
<code>BF16</code>	16	8 exp / 7 man	Google Brain Floating Point (matches FP32 dynamic range).
<code>8E</code>	8	4 exp / 3 man	OCP FP8 (E4M3) — NVIDIA Hopper & Blackwell AI training.
<code>8M</code>	8	5 exp / 2 man	OCP FP8 (E5M2) — NVIDIA Hopper & Blackwell AI inference.
<code>6E</code>	6	3 exp / 2 man	OCP Microscaling FP6 (E3M2) — Sub-byte quantized LLM weights.
<code>4E</code>	4	2 exp / 1 man	NVIDIA Blackwell NVFP4 (E2M1) — 4-bit AI tensor core weights.

All float decoders transcode through standard 64-bit IEEE 754 floats (`f64`) as a universal currency. You can quantize or transcode between any two floating-point formats in a single pass:

```
# Quantize FP16 weights directly to NVIDIA Hopper FP8:
bdd "16H → 8E" < model_fp16.bin > model_fp8.bin

# Quantize FP16 weights to NVIDIA Blackwell 4-bit NVFP4:
bdd "16H → 4E" < model_fp16.bin > model_nvfp4.bin
```

4. In-Flight Transformations & Field Fusion

Once fields are unpacked into a tuple, you can project, rearrange, bitwise-fuse, arithmetic-scale, and filter them before packing them to output.

1. Shorthand Tuple Projection: `{indices...}`

Use curly braces `{...}` to select, reorder, or duplicate fields:

- `{1, 0}` : Swaps field 0 and field 1 (e.g. `4U4U` $\rightarrow$ `{1, 0}` $\rightarrow$ `4U4U`).
- `{0, 2}` : Emits field 0 and field 2, discarding field 1.
- `{0, 0, 0}` : Duplicates field 0 three times.
- `{tag, length}` : Projects named fields directly.

2. Bitwise Field Fusion: `{0|1}` and `{0, 1|2}`

Often you need to glue multiple unaligned fields back into a single wider integer or bitfield. Use the pipe operator `|` :

- `{0|1}` : Glues field 0 and field 1 together into a single packed integer. If field 0 is `4U` (`1111` = `0xF`) and field 1 is `4U` (`1010` = `0xA`), `{0|1}` computes `(0xF << 4) | 0xA` = `0xFA` .
- `{0, 1|2}` : Keeps field 0 unchanged, and fuses field 1 and field 2 into a single field.
- `{tag, hi|lo}` : Keeps `tag` separate, and fuses `hi` and `lo` nibbles together.
- **Explicit Bit-Widths** (`{0:4|1:4}`) : Specify bit-lengths explicitly when operating on raw untyped tuples.

3. Ordered Math & Logic Manipulators

Chain manipulators directly inside arrow pipelines:

MANIPULATOR	SYNTAX	DESCRIPTION
Add	<code>add(val)</code> or <code>add(field, val)</code>	Adds <code>val</code> to field (default: field 0).
Subtract	<code>sub(val)</code> or <code>sub(field, val)</code>	Subtracts <code>val</code> from field.
Multiply	<code>mul(val)</code> or <code>mul(field, val)</code>	Multiplies field by <code>val</code> .
Divide	<code>div(val)</code> or <code>div(field, val)</code>	Performs integer division of field by <code>val</code> .
Modulo	<code>mod(val)</code> or <code>mod(field, val)</code>	Computes field modulo <code>val</code> .
Bitwise XOR	<code>xor(mask)</code> or <code>xor(field, mask)</code>	Bitwise XOR with hex (<code>0xAA</code>) or decimal mask.
Bitwise AND	<code>and(mask)</code> or <code>and(field, mask)</code>	Bitwise AND with mask.
Bitwise OR	<code>or(mask)</code> or <code>or(field, mask)</code>	Bitwise OR with mask.
Bitwise NOT	<code>not</code> or <code>not(field)</code>	Inverts all bits of the field.
Shift Left	<code>shift_left(bits)</code>	Left-shifts field by N bits.
Shift Right	<code>shift_right(bits)</code>	Right-shifts field by N bits (discards low N bits).
Clamp	<code>clamp(min, max:mode)</code>	Clamps value to <code>[min, max]</code> (<code>saturate</code> , <code>wrap</code> , <code>zero</code> , <code>drop</code>).
Round	<code>round(bits, mode)</code>	Rounds floats or integers (<code>trunc</code> , <code>floor</code> , <code>ceil</code> , <code>round</code>).
Filter	<code>filter(field, op, val)</code>	Drops entire record if predicate is false (<code>=</code> , <code>≠</code> , <code><</code> , <code>≤</code> , <code>></code> , <code>≥</code>).
Set	<code>set(val)</code> or <code>set(field, val)</code>	Directly assigns constant <code>val</code> to field (default: field 0).

5. Fascinating Real-World Scenarios

Here is `bdd` in action across eleven high-impact systems engineering and data pipeline workflows:

Scenario 1: IoT Sensor Calibration & Dynamic Range Compression

Embedded 12-bit ADCs (analog-to-digital converters) frequently stream raw readings offset by an analog DC bias. In this pipeline, we ingest 12-bit ADC samples, subtract the 512-count DC offset, amplify the sensor signal by $2\times$, clamp values to prevent DAC saturation (0..255), and pack the result into clean 8-bit DAC bytes:

```
bdd "12 → sub(512) → mul(2) → clamp(0, 255:saturate) → 8" < adc_raw.bin > dac_out.bin
```

Scenario 2: Real-Time Broadcast Video MPEG-TS Descrambling

In digital video broadcasting (DVB / ATSC), MPEG-2 Transport Streams transmit unaligned 188-byte packets. Packet headers contain sync bytes (`0x47`), a 13-bit PID, and scrambling control flags. In this pipeline, we isolate the 184-byte payload (`188B[32:1472]`), apply on-the-fly XOR descrambling against a rotating sync key (`0x5A`), and use `overwrite` to preserve the 4-byte transport header (`0x47` , PID, flags) intact while writing back the full 188-byte packets:

```
bdd "188B[32:1472] → xor(0x5A) → overwrite" < scrambled.ts > clear.ts
```

Scenario 3: Audio DSP Headroom Limiting & Dynamic Soft-Clipping

Digital studio masters recorded in 16-bit signed PCM (`16S` , range $-32768.. + 32767$) often require brickwall headroom limiting before transmission to prevent loudspeaker voice-coil blowout. This pipeline ingests 16-bit signed audio, clamps peaks strictly to $[-16000, +16000]$ with saturation, applies a 6 dB attenuation (`div(2)`), and repacks the output as bit-perfect signed PCM:

```
bdd "16S → clamp(-16000, 16000:saturate) → div(2) → 16S" < master.pcm > limited.pcm
```

Scenario 4: Network Packet QoS / DSCP & CoS Priority Rewriter

Network routers mark Quality of Service (QoS) inside the 8-bit Type of Service (ToS) field of IPv4 headers (RFC 791). The Differentiated Services Code Point (DSCP) occupies the upper 6 bits, while Explicit Congestion Notification (ECN) occupies the lower 2 bits. In this pipeline, we inspect IPv4 headers, isolate the ToS byte (`20B[8:8]`), rewrite the DSCP priority to Expedited Forwarding (`0x28` / EF Class) directly

using `set(0x28)` , and use `overwrite` to preserve all surrounding 19 bytes of the packet (IP addresses, packet length, TTL, payload) intact:

```
bdd "20B[8:8] → set(0x28) → overwrite" < incoming_packets.bin > qos_packets.bin
```

Scenario 5: Hardware Watchdog Heartbeat & Rolling Counter

Embedded watchdogs expect sequential rolling counter ticks where specific bits are periodically forced to a constant synchronization heartbeat to prove the micro-controller's ALU is alive and not stuck. This pipeline generates an infinite hardware counter, slices each byte into high and low 4-bit nibbles (`4U4U`), sets the high nibble directly to constant `0xF` using `set(0, 0xF)` , fuses the nibbles back together (`{0|1}`), and emits formatted hexadecimal words to stdout:

```
bdd "counter → 8 → 4U4U → set(0, 0xF) → {0|1} → hex" --count 4
# Output: f0 f1 f2 f3
```

Scenario 6: Direct AI Model Quantization (FP16 → Hopper FP8 / Blackwell NVFP4)

Quantizing large neural network checkpoints usually requires installing gigabytes of heavy Python frameworks (`torch` , `transformers` , `accelerate`). With `bdd` , you can quantize raw FP16 weights directly to NVIDIA Hopper OCP FP8 or NVIDIA Blackwell 4-bit NVFP4 at line-rate multi-gigabit speeds using zero Python dependencies:

```
# Quantize half-precision FP16 weights to NVIDIA Hopper FP8 (E4M3):
bdd "16H → 8E" < weights_fp16.bin > weights_fp8.bin

# Quantize half-precision FP16 weights to NVIDIA Blackwell 4-bit NVFP4 (E2M1):
bdd "16H → 4E" < weights_fp16.bin > weights_nvfp4.bin
```

Scenario 7: In-Place MPEG-TS Header Patching with Real-Time Stream Tapping

In broadcast transmission lines, video engineers frequently need to remap unaligned 13-bit PIDs (Packet Identifiers) on live transport stream packets while saving an audit log of the extracted original values. This pipeline extracts the 13-bit PID from 188-byte containers (`188B[11:13]`), mirrors the original PIDs directly to an audit file using `tee('pids_audit.bin')` , XOR-patches the PID value, and uses `overwrite` to write back the entire 188-byte packet with all original sync bytes (`0x47`), payload data, and timing intact:

```
bdd "file('input.ts') → 188B[11:13] → tee('pids_audit.bin') → xor(0x1FFF) → overwrite → file('patched.ts')"
```

Scenario 8: Zero-Copy Network Protocol Dissection & UDP Telemetry Filtering

Raw Ethernet captures and pcap records deliver network traffic as dense packet streams. Using format presets, `bdd` unpacks protocol headers without manual bit-offset code. In this pipeline, we inspect incoming packets using the built-in `ipv4-header` preset, filter for UDP datagrams (`protocol = 17` , field index 9), and output structured JSON telemetry:

```
bdd "file('traffic.bin') → ipv4-header → filter(9, =, 17) → json"
# Output:
{"checksum":23700,"dscp":0,"dst_ip":16843009,"ecn":0,"flags":2,"frag_offset":0,"id":6699,"ihl":5,"protocol":17,"src_ip":3232235876,"total_length":32,"ttl":64,"version":4}
```

Scenario 9: Binary Firmware Heuristics & Visual Entropy Auditing

When reverse-engineering unknown firmware blobs, ROM dumps, or encrypted containers, security analysts need to determine data layouts without running full disassembly. With `--probe` and `--probe-visual` , `bdd` scans the binary to measure Shannon entropy (0.0..8.0 bits/byte), auto-correlate periodic record strides (e.g. 40-byte telemetry frames or 188-byte MPEG packets), and render an ANSI heat-map sparkline:

```
bdd --probe firmware.bin --probe-visual
```

Output:

```
Target:                firmware.bin
Sample Size:           130 bytes (1040 bits)
Shannon Entropy:       5.0170 / 8.0000 bits/byte
Entropy Sparkline:     [  █  ████ ████ ████ ████ ████ ████ ████ ████ ] (26 blocks)
Entropy Level:         Medium (4.00-6.50): Structured data, mixed binary headers and text
                        payloads
General Diagnosis:     Structured binary stream with periodic record stride of 40 bytes (320
                        bits).
```

Byte Class Distribution:

- Null Bytes (0x00): 31 (23.85%)
- Printable ASCII: 57 (43.85%)
- High Bytes ($\geq 0x80$): 20 (15.38%)

Detected Periodic Strides (Autocorrelation):

- ```

 • 40 bytes (320 bits) [Match: 30.00%, Conf: High]: Periodic structured record
boundary

```

Identified ASCII String Runs:

- Offset 0x0070 (len 14): "GET / HTTP/1.1"

Visual Entropy Map (Shannon bits/byte distribution):

```
0x000000 [130B] [...] 5.02 H
```

## Scenario 10: Multi-Source Clock Synthesis & Stream Interleaving

Embedded telemetry frequently requires interleaving a monotonically increasing hardware clock tick with sensor payloads or zero-padded sync bytes. Using `bdd`'s bracket syntax `[ source1, source2, ... ]`, multiple stream generators are multiplexed into a single stream:

```
Interleave a 16-bit hardware counter with 16-bit zero-padded channels:
bdd "[counter, zeros] → 16 → hex" --count 4
Output: 0000 0000 0001 0000 0002 0000 0003 0000
```

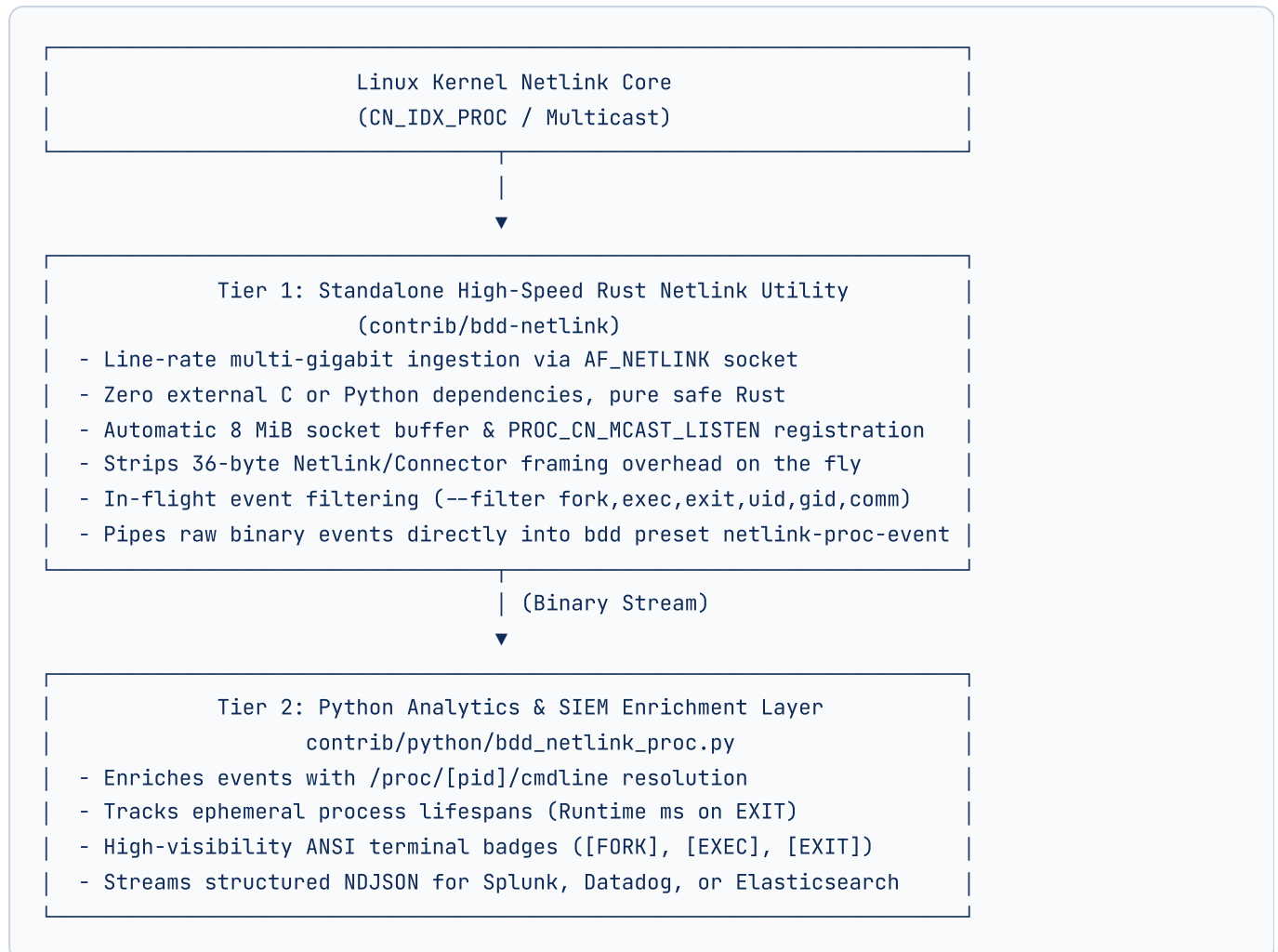
### Scenario 11: Machine Code Instruction Field Disassembly (RISC-V)

Microprocessor architectures pack instruction opcodes, registers, and function codes into sub-byte bitfields. In this pipeline, we ingest 32-bit RISC-V binary machine words, unpack them according to the `riscv-r-type` schema ( `funct7:7u`, `rs2:5u`, `rs1:5u`, `funct3:3u`, `rd:5u`, `opcode:7u` ), and output structured JSON instruction representations:

```
printf "\x33\x02\x42\x00" | bdd "riscv-r-type → json"
Output: {"funct3":1,"funct7":76,"opcode":0,"rd":4,"rs1":4,"rs2":1}
```

## 6. Linux Netlink Kernel Telemetry & Process Lifecycles

Modern Linux kernels provide real-time process execution, fork, exit, and credential events via the Netlink Process Connector ( `AF_NETLINK` , `CN_IDX_PROC` ). `bdd` provides a two-tier architecture for kernel telemetry:



### 1. Tier 1: Standalone Rust Kernel Ingestion ( `contrib/bdd-netlink` )

Run `bdd-netlink` with root privileges to connect directly to the kernel process connector and pipe into `bdd` :

```
Ingest kernel process events directly into structured JSON:
sudo ./contrib/bdd-netlink | bdd --preset netlink-proc-event --output-json

Unpack specific proc_event header fields (Timestamp, CPU ID, Event Code) using pattern
arrows:
sudo ./contrib/bdd-netlink | bdd "netlink-proc-event → visual"

Filter specifically for process execution events (exec):
sudo ./contrib/bdd-netlink --filter exec | bdd --preset netlink-proc-event --json-object
```

## 2. Tier 2: Python Analytics & SIEM Enrichment ( `bdd_netlink_proc.py` )

While `bdd-netlink` operates as an ultra-high-speed, zero-copy packet ingestion engine, security operations centers (SOC) and site reliability engineers (SRE) require stateful correlation and command-line resolution:

```
Launch interactive terminal monitor with high-visibility ANSI badges:
sudo python3 contrib/python/bdd_netlink_proc.py

Filter only process execution events (EXEC) in real time:
sudo python3 contrib/python/bdd_netlink_proc.py --event EXEC

Stream machine-readable NDJSON for SIEM ingestion:
sudo python3 contrib/python/bdd_netlink_proc.py --json
```

---

## 7. Sub-Byte Bitstream Mechanics: Operating Across Byte Boundaries

---

A natural question when dealing with bitstream data is: **Can `bdd` handle streams and units that are not aligned to 8-bit byte boundaries?**

**Yes, completely.** `bdd` is built from the ground up as a sub-byte bitstream processor. Alignment to 8-bit byte boundaries is never required:

- 1. Arbitrary Bit Offsets:** An input stream can begin at any bit offset directly in pattern syntax ( `N : unit` , e.g., `11 : 13` skips 11 bits and begins streaming 13-bit units across byte boundaries, or via `--input-skip-bits=N` ).
  - 2. Dense Sub-Byte & Odd-Width Units:** Units of any bit width (e.g., 1-bit flags, 3-bit opcodes, 5-bit fields, 12-bit ADC readings, 13-bit PIDs) pack continuously across byte boundaries without inter-unit padding. For instance, eight 3-bit units ( `3U` ) pack tightly into exactly 3 bytes (24 bits).
  - 3. Periodic Bit Gaps:** You can skip arbitrary unaligned bit gaps between units directly in pattern syntax ( `unit + gap` , e.g., `8 + 24` skips 24 bits after every byte, or via `--input-gap=24` ).
  - 4. End-of-Stream (EOF) Alignment Controls:**
    - **Default Zero-Padding on Output:** Unix files, fifos, and pipes operate strictly on 8-bit bytes. When writing out an unaligned bitstream (e.g., 13 bits total), `bdd` zero-pads the remaining bits of the final byte ( `101...000` ) so that POSIX byte writes remain valid.
    - `--drop-partial-eof` : When reading an input stream whose final unit is truncated (e.g., only 5 bits remaining when 12 bits are expected), `--drop-partial-eof` discards the incomplete trailing bits instead of zero-padding them into a spurious final unit.
    - `--input-assert-aligned` : Strictly verifies that the input stream terminates exactly on an 8-bit byte boundary, aborting with exit code 1 ( `Non-aligned end of file` ) if stray unaligned bits remain at EOF.
-

## 8. Multi-Stream Coordination: Merging, Interleaving, Tapping & Demuxing

---

### Round-Robin Stream Merging

`bdd` can interleave data from multiple streams round-robin using declarative arrow syntax or CLI flags:

```
Declarative bracket syntax:
bdd "[file('payload.bin'):8, file('addresses.bin'):32] → hex"

In-pipeline interleave operator with zero-padding on premature EOF:
bdd "file('payload.bin'):8 → interleave(file('addresses.bin'):32, pad:zeros) → hex"

Equivalent classical CLI flags (for automated scripts or programmatic drivers):
bdd --input-file=payload.bin --input-unit=8 \
 --merge-file=addresses.bin --merge-unit=32 \
 --output-hex
```

### Non-Destructive Stream Tapping ( `tee` )

Mirror any intermediate bitstream to a secondary file without interrupting the downstream pipeline:

```
Intercept pre-quantized 16-bit audio while continuing to emit 8-bit compressed audio:
bdd "file('master.pcm') → 16S → clamp(-16000, 16000:saturate) → tee('audit.pcm') → div(2) -
 > 8S → stdout"
```

### Channel Demuxing & Splitting ( `--demux` , `--demux-files` )

Split multi-channel streams into separate physical files in a single pass:

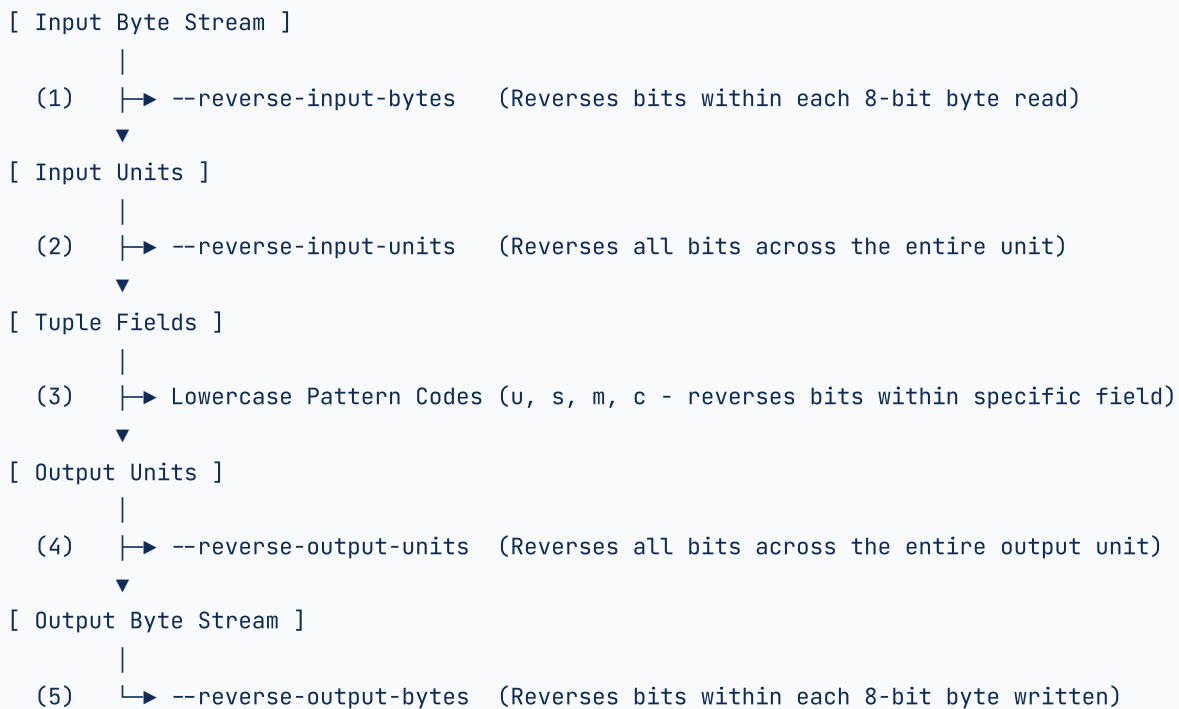
```
Slices 32-bit stereo audio into Left and Right channel files:
bdd "16S16S" --demux 0:left.raw --demux 1:right.raw < stereo.raw
```

---

## 9. Taming Bit Reversals & Endianness

---

Bit reversals across arbitrary bit widths frequently cause confusion. `bdd` cleanly isolates reversals into four distinct operational layers:



- `--input-little-endian` : Shorthand combining `--reverse-input-bytes` and `--reverse-input-units`.
- `--output-little-endian` : Shorthand combining `--reverse-output-bytes` and `--reverse-output-units`.

## 10. Exabyte Scale, Fast Hardware Seeking & Size Expressions

Can `bdd` skip over gigabytes of data on the command line? **Yes, without limit:**

- **Full 64-Bit Architecture ( `u64` )**: All skips, offsets, gaps, units, and record counts are represented internally as 64-bit unsigned integers. `bdd` can represent bit offsets up to  $2^{64} - 1 \approx 1.84 \times 10^{19}$  bits, which equals **2.3 Exabytes** (2,305,843,009 Gigabytes).
- **Filesystem Seek Limits**: On seekable files, Linux `lseek64` supports offsets up to  $2^{63} - 1$  bytes = **9.22 Exabytes**, executed in  $O(1)$  constant time (microseconds) without memory overhead.
- **Human-Friendly Size Suffixes**: All numeric arguments accept standard scale suffixes:
  - **Binary multiples**: `K`, `M`, `G`, `T`, `P`, `E` (or `Ki`, `Mi`, `Gi`, `Ti`, `Pi`, `Ei`).
  - **Decimal multiples**: `KB`, `MB`, `GB`, `TB`, `PB`, `EB`.
  - **Byte-scaled bit skips**: On bit options ( `--input-skip-bits`, `--merge-skip-bits` ), specifying `B` (e.g. `10GiB`, `4GB`, `100B` ) automatically multiplies bytes by 8 bits.
- **Multiplication Expressions**: All size and count options support multiplication expressions using `*` or `x`:
  - **2D/3D dimensions and frame strides**: `--input-skip-bits=1920x1080*24` or `--input-skip-bits="1920 * 1080 * 3B"`.
  - **Sizing units and offsets**: `--input-unit=3*8` (24-bit unit) or `--input-offset=2*8` (16-bit offset).

```
Instantly seek 10 GiB into a raw disk image and inspect 4 bytes (32 bits):
bdd "file('disk.img') : 10GiB : 32 → hex" --count 1
(or using explicit flags: bdd --input-file=disk.img --input-skip-bits=10GiB --count=4 --output-hex)

Skip a 1080p 24-bit uncompressed RGB frame buffer and inspect the first pixel:
bdd "file('video.raw') : 1920x1080*24 : 24 → hex" --count 1
```

## 11. Discovery, AI Ergonomics & Web GUI

`bdd` provides native primitives for autonomous AI coding agents, reverse engineers, and pipeline automation:

### Format Presets ( `--preset` , `--list-presets` , `--presets-file` )

Instead of manually typing complex bit patterns, format presets configure patterns, unit widths, and field names in one step.

Presets are stored in a standalone `presets.json` file and compiled directly into `bdd` as embedded defaults.

`bdd` automatically discovers preset files using the following resolution hierarchy:

1. Explicit CLI argument: `--presets-file <PATH>`
2. Environment variable: `BDD_PRESETS_FILE=<PATH>`
3. Current working directory: `./presets.json`
4. User configuration directory: `&approx;/.config/bdd/presets.json`
5. System FHS directories (packaged with `.deb` and `.rpm`): `/usr/share/bdd/presets.json` , `/usr/local/share/bdd/presets.json` , or `/etc/bdd/presets.json`
6. Embedded offline defaults compiled directly into the binary.

```
List all available presets:
bdd --list-presets

Use an alternate local presets file:
bdd --presets-file ./my_presets.json --list-presets
bdd --presets-file ./my_presets.json --preset custom-header
```

Standard presets include: `mp3-header` , `mpeg-ts` , `wav-header` , `jpeg-sof0` , `h264-nal` , `nvfp4` , `fp6-e3m2` , `fp8-e4m3` , `fp8-e5m2` , `bf16` , `fp16` , `ipv4-header` , `udp-header` , `tcp-header` , `riscv-r-type` , `proc-pagemap` , `proc-auxv` , `pci-config` , `netlink-proc-event` .

```
Inspect MPEG-TS headers as structured JSON objects directly via pattern pipeline:
bdd "file('stream.ts') → 188B : mpeg-ts → json:object" --count 3
(or using explicit flags: bdd --input-file stream.ts --input-raw-unit 188B --preset mpeg-ts -
-json-object --count 3)
```

## Pattern Explainer & Struct Code Generation ( `--explain-pattern` )

Inspect bit layouts or generate production-ready packed C structs and Rust struct definitions via decoupled contrib translators:

```
Generate packed C struct definition with bitfields:
bdd --explain-pattern "sync:8u,pid:13u" --output-json | python3 contrib/python/json_to_c.py --
name MpegTsHeader

Generate Rust struct with #[repr(C, packed)]:
bdd --explain-pattern "sync:8u,pid:13u" --output-json | python3 contrib/python/json_to_rust.py
--name MpegTsHeader

Core bdd also supports direct --export-c and --export-rust flags:
bdd --preset mpeg-ts --export-c
bdd --preset mpeg-ts --export-rust
```

## Binary Prober ( `--probe` ) & Cryptographic Key Discovery ( `--probe-keys` )

Analyze unknown binary blobs without prior schema knowledge:

- `--probe` : Computes Shannon entropy (0..8 bits/byte), byte class distributions, and auto-correlates periodic strides (1..512 bytes).
- `--probe-visual` : Renders 2D terminal ANSI heatmaps and sparklines (  ).
- `--probe-keys` : Scans the bitstream using sliding-window entropy analysis to locate candidate cryptographic keys (AES-128, AES-256, ChaCha20, Ed25519) embedded inside code or padding:

```
bdd --input-file=firmware.bin --probe-keys=256
```

## Model Context Protocol (MCP) Server ( `bdd-mcp` / `--mcp` )

The Model Context Protocol server is decoupled into a standalone companion crate and binary ( `crates/bdd-mcp` ) in the workspace, isolating protocol churn from the core bitstream library. It provides a standard JSON-RPC 2.0 stdio interface enabling AI assistants (Antigravity, Claude Desktop, Cursor) to slice, unpack, inspect, and probe binary streams:

```
Launch standalone binary directly:
bdd-mcp

Or launch via cargo:
cargo run -p bdd-mcp

Or via bdd CLI delegation:
bdd --mcp
```

Tools exposed: `bdd_slice`, `bdd_probe`, `bdd_probe_units`, `bdd_transcode`, `bdd_generate`, `bdd_explain_pattern`, `bdd_list_presets`.

## Interactive Web Application ( `web/` )

The interactive web UI application is decoupled into the `web/` directory and interacts with `bdd` exclusively via the CLI without embedding a web server inside the core bitstream binary:

```
Launch via Python 3 (zero external dependencies)
python3 web/server.py [port]

Or launch via standalone Rust server
cargo run --manifest-path web/Cargo.toml -- [port]

Or via Makefile shortcut
make web
```

Features drag-and-drop file inspection, live visual bit breakdown, color-coded pattern layouts, real-time CLI command generation, and multi-sink inspection.

---

## 12. Programmatic Bitstream SDKs (Rust, C & Python)

When building device drivers, multimedia decoders, or embedded firmware, you often need to manipulate unaligned bitstreams **directly in memory** without creating files, pipes, or shell subprocesses.

### Does `pip install bdd` Contain the Binary?

**Yes.** The Python package is fully self-contained:

- It includes pure-Python implementations of `BitStreamReader` and `BitStreamWriter` that run anywhere without a compiler.
- It automatically locates and loads the compiled high-performance Rust library ( `libbdd.so` ) via `ctypes` when available, seamlessly accelerating throughput to multi-gigabit speeds.

## Rust Library API ( `bdd::bits` )

Add `bdd` to your `Cargo.toml` :

```
[dependencies]
bdd = { git = "https://github.com/e-t-u/bdd.git" }
```

```
use bdd::bits::{BitStreamReader, BitStreamWriter, read_bits_u64, write_bits_u64};

fn main() → Result<(), Box<dyn std::error::Error>> {
 // 1. Reading unaligned bits across byte boundaries:
 // Extract 13-bit MPEG PID starting at bit offset 11:
 let packet = [0x47, 0x1F, 0xFF, 0x10];
 let pid = read_bits_u64(&packet, 11, 13)?;
 assert_eq!(pid, 0x1FFF);

 // 2. Sequential in-memory bitstream reading:
 // Raw 3-byte payload holding two dense 12-bit ADC samples (0xABC, 0xDEF):
 let raw_adc = [0xAB, 0xCD, 0xEF]; // 24 bits
 let mut reader = BitStreamReader::new(&raw_adc);
 let s0 = reader.read_bits(12)?; // 0xABC (2748)
 let s1 = reader.read_bits(12)?; // 0xDEF (3567)
 assert!(reader.is_empty());

 // 3. Building an unaligned bitstream with BitStreamWriter:
 let mut writer = BitStreamWriter::new();
 writer.write_bits(0b101, 3)?; // 3-bit status flag
 writer.write_bits(s0, 12)?; // 12-bit sensor sample
 writer.write_bits(1, 1)?; // 1-bit parity flag

 let (bytes, total_bits) = writer.finish();
 assert_eq!(total_bits, 16);
 assert_eq!(bytes.len(), 2);

 // 4. In-place bitfield mutation without altering neighbor bits:
 let mut header = [0xFF, 0x00];
 write_bits_u64(&mut header, 6, 4, 3)?; // write 0b0011 at bit offset 6
 assert_eq!(header[0], 0xFC);
 assert_eq!(header[1], 0xC0);

 Ok(())
}
```

---

## Statistical Metric Analysis & Accumulator SDK ( `bdd::analysis` )

`bdd` provides an extensible, zero-dependency metric engine in `bdd::analysis` for information-theoretic analysis, statistical reductions, and multi-column stream profiling:

## 1. Available Metric Accumulators

| ACCUMULATOR                        | CANONICAL NAME(S)                                       | DESCRIPTION                                                        | RETURN TYPE                                         |
|------------------------------------|---------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------|
| <code>CountAccumulator</code>      | <code>"count"</code>                                    | Counts total elements ingested                                     | <code>Field::UInt</code>                            |
| <code>SumAccumulator</code>        | <code>"sum"</code>                                      | Running arithmetic sum ( <code>BigInt</code> or <code>f64</code> ) | <code>Field::Int</code> / <code>Field::Float</code> |
| <code>MinAccumulator</code>        | <code>"min"</code>                                      | Running minimum element                                            | <code>Field</code>                                  |
| <code>MaxAccumulator</code>        | <code>"max"</code>                                      | Running maximum element                                            | <code>Field</code>                                  |
| <code>MeanAccumulator</code>       | <code>"avg"</code> , <code>"mean"</code>                | Running arithmetic mean ( $\sum x/N$ )                             | <code>Field::Float</code>                           |
| <code>EntropyAccumulator</code>    | <code>"entropy"</code> , <code>"shannon_entropy"</code> | Cumulative byte-level Shannon entropy ( 0.0..8.0 bits/byte)        | <code>Field::Float</code>                           |
| <code>BitBalanceAccumulator</code> | <code>"balance"</code> , <code>"bit_balance"</code>     | Percentage of set 1 -bits ( 0.0%..100.0% )                         | <code>Field::Float</code>                           |
| <code>VarianceAccumulator</code>   | <code>"variance"</code> , <code>"stddev"</code>         | Online sample variance/stddev via Welford's algorithm              | <code>Field::Float</code>                           |
| <code>DistinctAccumulator</code>   | <code>"distinct"</code> , <code>"unique"</code>         | Cardinality count of unique field values                           | <code>Field::UInt</code>                            |

## 2. Evaluating Accumulators & Direct Field Analysis

```
use bdd::analysis::{create_accumulator, Accumulator};
use bdd::Field;

fn main() → Result<(), Box<dyn std::error::Error>> {
 // Direct analysis methods on any sliced Field:
 let payload = Field::Bytes(vec![0xAA; 32]);
 println!("Shannon entropy: {:.4} bits/byte", payload.shannon_entropy()); // 0.0000
 println!("Bit balance: {:.2}% ones", payload.bit_balance()); // 50.00%
 println!("Hamming weight: {} set bits", payload.hamming_weight()); // 128

 // Evaluating metric accumulators dynamically or via concrete structs:
 let mut entropy = create_accumulator("entropy")?;
 let mut balance = create_accumulator("balance")?;
 let mut avg = create_accumulator("avg")?;

 let samples = vec![
 Field::UInt(10u32.into()),
 Field::UInt(20u32.into()),
 Field::UInt(30u32.into()),
];

 for s in &samples {
 entropy.update(s);
 balance.update(s);
 avg.update(s);
 }

 println!("Cumulative Entropy: {}", entropy.value());
 println!("Bit Balance: {}", balance.value());
 println!("Arithmetic Mean: {}", avg.value()); // 20.0

 Ok(())
}
```

## 3. Multi-Column Stream Profiling ( TupleCollector )

`TupleCollector` profiles multi-field tuple streams across columns and renders formatted tables or JSON summaries:

```

use bdd::analysis::TupleCollector;
use bdd::Field;

let schema = vec!["sync".to_string(), "pid".to_string(), "payload".to_string()];
let mut profiler = TupleCollector::new(&schema);

// Ingest tuples as they flow through your streaming pipeline:
profiler.update(&[
 Field::UInt(0x47u32.into()),
 Field::UInt(256u32.into()),
 Field::Bytes(vec![0xAA; 184]),
]);

// Emit an aligned ASCII summary report:
println!("{}", profiler.format_table());

```

## Standalone Small Floating-Point Crate ( [bdd-small-floats](#) )

For projects requiring specialized AI, GPU, and sub-byte floating-point codecs without pulling in the full bitstream engine, the encoders and decoders are available as an independent, zero-dependency subcrate located in [crates/bdd-small-floats](#) :

```

[dependencies]
bdd-small-floats = { path = "crates/bdd-small-floats" }

```

```

use bdd_small_floats::{
 decode_f16, encode_f16,
 decode_bf16, encode_bf16,
 decode_fp8_e4m3, encode_fp8_e4m3,
 decode_fp8_e5m2, encode_fp8_e5m2,
 decode_fp6_e3m2, encode_fp6_e3m2,
 decode_fp4_e2m1, encode_fp4_e2m1,
};

fn main() {
 // NVIDIA Blackwell / OCP FP4 E2M1 (4-bit):
 let fp4 = encode_fp4_e2m1(1.5);
 assert_eq!(decode_fp4_e2m1(fp4), 1.5);

 // OCP FP8 E4M3FN (8-bit):
 let fp8 = encode_fp8_e4m3(1.0);
 assert_eq!(decode_fp8_e4m3(fp8), 1.0);

 // IEEE 754 Half-Precision FP16 (16-bit):
 let fp16 = encode_f16(2.0);
 assert_eq!(decode_f16(fp16), 2.0);
}

```

---

## C / C++ API ( [include/bdd.h](#) )

Link against `libbdd.so` :

```

#include "bdd.h"
#include <stdio.h>
#include <assert.h>

int main(void) {
 uint8_t packet[4] = { 0x47, 0x1F, 0xFF, 0x10 };
 uint64_t pid = 0;

 // Read 13-bit PID at bit offset 11 across byte boundaries:
 bdd_read_bits_u64(packet, sizeof(packet), 11, 13, &pid);
 printf("Extracted 13-bit PID: 0x%lX\n", (unsigned long)pid);

 // Write a 5-bit priority tag at bit offset 27 without touching other bits:
 bdd_write_bits_u64(packet, sizeof(packet), 27, 5, 0x1F);

 // Unpack multi-field structured tuple directly from memory:
 uint64_t fields[4];
 uint8_t telemetry[2] = { 0xAB, 0xCD };
 int count = bdd_unpack_buffer("3U1x2u3M", telemetry, sizeof(telemetry), 0, fields, 4);
 printf("Unpacked %d fields: [%lu, %lu, %lu, %lu]\n",
 count, fields[0], fields[1], fields[2], fields[3]);

 return 0;
}

```

## Python SDK ( `pip install bdd` )

```
import bdd

1. Unaligned buffer slicing:
data = b"\xAC\xF0" # 10101100 11110000
val = bdd.read_bits(data, bit_offset=6, bit_count=4)
print("Unaligned slice:", val) # → 3

2. Sequential in-memory bitstream reading:
raw_adc = b"\xAB\xCD\xEF" # two dense 12-bit samples
reader = bdd.BitStreamReader(raw_adc)
sample0 = reader.read(12) # 0xABC (2748)
sample1 = reader.read(12) # 0xDEF (3567)
assert reader.is_empty()

3. Packing unaligned bitstreams with BitStreamWriter:
writer = bdd.BitStreamWriter()
writer.write(0b101, bits=3) # 3-bit status
writer.write(0xABC, bits=12) # 12-bit sensor reading
writer.write(1, bits=1) # 1-bit parity
packed_bytes, exact_bits = writer.finish()
print(f"Packed {exact_bits} bits into {len(packed_bytes)} bytes: {packed_bytes.hex()}")
→ Packed 16 bits into 2 bytes: b579

4. Direct in-memory pattern unpacking:
fields = bdd.unpack_buffer("3U2u3M", b"\xF5", bit_offset=0)
print("Unpacked fields:", fields) # → (7, 1, 1, 3)

5. Native AI float format conversions:
val = bdd.get_bdd().decode_f16(0x3C00) # FP16 → 1.0
bits = bdd.get_bdd().encode_fp4(1.0) # 1.0 → NVFP4
```

---

## 13. Architecture, Performance & Benchmarks

Measured via `make bench` ( `benches/throughput.rs` ) on Linux x86\_64:

| OPERATION                            | TOTAL VOLUME | THROUGHPUT (BITS/S) | THROUGHPUT (BYTES/S) | NOTES                                          |
|--------------------------------------|--------------|---------------------|----------------------|------------------------------------------------|
| Hardware 64-bit Bit Reversal         | 128 Mbits    | 16.00 Gbps          | 2,000 MB/s           | Direct CPU<br><code>u64::reverse_bits()</code> |
| Bignum 1024-bit Packing/Unpacking    | 10.2 Mbits   | 5.27 Gbps           | 658.6 MB/s           | Large-block bignum bitfield packing            |
| Bignum 256-bit Packing/Unpacking     | 12.8 Mbits   | 1.90 Gbps           | 237.2 MB/s           | SHA-256 size field packing/unpacking           |
| Synthetic 8-bit Linear Stream        | 16.0 Mbits   | 234.5 Mbps          | 29.3 MB/s            | Continuous bit generation & sink flush         |
| 1-bit Single-Bit Resolution Stream   | 0.5 Mbits    | 43.3 Mbps           | 5.4 MB/s             | Single-bit slice accumulation & packing        |
| Bignum 1024-bit Bit Reversal         | 5.1 Mbits    | 39.4 Mbps           | 4.9 MB/s             | Full arbitrary-precision bit reversal          |
| Unaligned 3-bit to 8-bit Extraction  | 3.0 Mbits    | 39.0 Mbps           | 4.9 MB/s             | Cross-byte boundary accumulation               |
| Tuple Pipeline ( 2U3U3U → Rearrange) | 2.0 Mbits    | 29.7 Mbps           | 3.7 MB/s             | Multi-field unpack, reorder & repack           |

## 14. Command Line Options Reference (Secondary Reference)

### Why Two Interfaces? The Machine-Level Substrate Underneath Patterns

While human engineers and reverse engineers think intuitively in **declarative pattern pipelines** ( `"source → container → pattern → manipulators → sink"` ), automated systems, autonomous AI agents (such as LLMs generating shell commands), CI/CD pipelines, and script wrappers often prefer explicit, granular command-line arguments.

`bdd` is architected so that **every single pattern manipulation maps 1:1 to an underlying CLI option**:

| PATTERN SYNTAX OPERATOR                                  | EQUIVALENT EXPLICIT CLI FLAG                                                        | PURPOSE                                   |
|----------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------|
| <code>file('data.bin')</code>                            | <code>--input-file=data.bin</code>                                                  | Primary file input                        |
| <code>skip : ...</code>                                  | <code>--input-skip-bits=N</code>                                                    | Initial stream bit offset / hardware seek |
| <code>raw[offset:unit]</code>                            | <code>--input-raw-unit=R --input-offset=0 --input-unit=U</code>                     | Container slice framing                   |
| <code>... + gap</code>                                   | <code>--input-gap=G</code>                                                          | Periodic inter-unit gap                   |
| <code>4U4U</code> , <code>16H</code> , etc.              | <code>--input-pattern="4U4U" , --preset=nvfp4</code>                                | Field tuple unpacking                     |
| <code>{1, 0}</code> , <code>{0 1}</code>                 | <code>--rearrange="1,0" , --rearrange="0 1"</code>                                  | Field permutation & bitwise fusion        |
| <code>add(val)</code> , <code>mul(val)</code>            | <code>--add=val , --mul=val</code>                                                  | Arithmetic scaling                        |
| <code>xor(mask)</code> , <code>not</code>                | <code>--xor=mask , --not</code>                                                     | Bitwise masking & logic                   |
| <code>clamp(min, max:mode)</code>                        | <code>--clamp="min,max:mode"</code>                                                 | Value saturation / bounding               |
| <code>filter(field, op, val)</code>                      | <code>--filter="field,op,val"</code>                                                | Record predicate filtering                |
| <code>set(val)</code> , <code>set(f, val)</code>         | <code>--set="f,val"</code>                                                          | Assign constant value to field            |
| <code>overwrite</code>                                   | <code>--overwrite (or --as-is)</code>                                               | In-place container editing                |
| <code>[ s1, s2 ]</code> , <code>interleave</code>        | <code>--merge-file=s2 --merge-unit=...</code>                                       | Round-robin multi-stream interleaving     |
| <code>pad:zeros</code>                                   | <code>--merge-pad-zeros</code> (default stream behavior)                            | Zero-padding at secondary EOF             |
| <code>tee('path')</code>                                 | <code>--tee="path"</code>                                                           | Intermediate non-destructive tapping      |
| <code>hex</code> , <code>bits</code> , <code>json</code> | <code>--output-hex</code> , <code>--output-bits</code> , <code>--output-json</code> | Output formatting sinks                   |

This makes `bdd` ideal for **autonomous LLM coding agents**: agents can either generate clean high-level stream pipelines or assemble robust programmatic flags with zero ambiguity.

Usage: bdd [OPTIONS] [STREAM\_PATTERN]

Arguments:

[STREAM\_PATTERN] Stream I/O pipeline or pattern (e.g. '8→3', '4U4U → {0|1} → hex', '188B[11:13] → 13', '4U4U')

Input Unit & Container Options:

-p, --input-pattern <PATTERN> Bit pattern to unpack input (e.g. "3U1x2u3M")  
-u, --input-unit <BITS> Size of active unit in bits (default: 8, or auto-inferred from pattern)  
--preset <NAME> Use built-in protocol or float preset (e.g. mp3-header, mpeg-ts, nvfp4)  
--list-presets List all built-in format presets and exit  
--presets-file <PATH> Path to custom presets JSON file (overrides default presets path)  
--input-skip-bits <BITS> Initial bit offset (skip) before first container [default: 0]  
--input-skip-units <UNITS> Skip initial N containers from input stream [default: 0]  
--input-gap <BITS> Bit gap skipped between repeating containers (or between units) [default: 0]  
--input-raw-unit <BITS> Size of repeating container in bits  
--input-offset <BITS> Bit offset (pregap) of unit inside container [default: 0]  
--input-assert-aligned Error if EOF is not byte-aligned  
--drop-partial-eof Discard incomplete trailing bits at EOF instead of zero-padding  
--no-seek, --do-not-seek Disable seeking on all inputs (force streaming sequential read)  
--input-use-seek Explicitly enable seeking on input (default: true if seekable)  
--no-mmap, --do-not-mmap Disable memory-mapped I/O (force standard buffered reads)  
--mmap Explicitly enable memory-mapped I/O on input files (default: auto)

Synthetic Stream Sources & Telemetry:

-c, --input-counter Generate sequential counter numbers (0, 1, 2...)  
-0, --input-zeros Generate endless stream of zero bits  
-1, --input-ones Generate endless stream of one bits  
-r, --input-random Generate random bits from /dev/urandom  
-t, --input-tuples Read comma-separated tuple lines from text input  
--skip <UNITS> Skip initial N units [default: 0]  
--count <COUNT> Process at most N units (0 or omitted = infinite) [default: 0]

Bit Reversal Options:

--reverse-input-bytes Reverse bit order within each input byte  
--reverse-input-units Reverse bit order across entire input unit  
--input-little-endian Combined byte and unit reversal for input  
--reverse-output-bytes Reverse bit order within each output byte  
--reverse-output-units Reverse bit order across entire output unit  
--output-little-endian Combined byte and unit reversal for output

## Tuple Manipulators:

|                                             |                                                                                                                                                      |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--rearrange &lt;FIELDS&gt;</code>     | Explicit output field order by input index (e.g. "1,0", "0 1", "{0, 1 2}")                                                                           |
| <code>--round &lt;F,LIMIT[,MODE]&gt;</code> | Round or clamp field F (alias: <code>--cut-maxint</code> )                                                                                           |
| <code>--remove-right &lt;F,BITS&gt;</code>  | Right-shift field F by BITS                                                                                                                          |
| <code>--shift-right &lt;F,BITS&gt;</code>   | Right-shift field F by BITS (synonym)                                                                                                                |
| <code>--shift-left &lt;F,BITS&gt;</code>    | Left-shift field F by BITS                                                                                                                           |
| <code>--xor &lt;F,PARAM&gt;</code>          | Bitwise XOR field F with PARAM                                                                                                                       |
| <code>--and &lt;F,PARAM&gt;</code>          | Bitwise AND field F with PARAM                                                                                                                       |
| <code>--or &lt;F,PARAM&gt;</code>           | Bitwise OR field F with PARAM                                                                                                                        |
| <code>--not &lt;FIELD&gt;</code>            | Bitwise NOT (inverts bits of field F)                                                                                                                |
| <code>--add &lt;F,PARAM&gt;</code>          | Add PARAM to field F                                                                                                                                 |
| <code>--sub &lt;F,PARAM&gt;</code>          | Subtract PARAM from field F                                                                                                                          |
| <code>--mul &lt;F,PARAM&gt;</code>          | Multiply field F by PARAM                                                                                                                            |
| <code>--div &lt;F,PARAM&gt;</code>          | Integer division of field F by PARAM                                                                                                                 |
| <code>--mod &lt;F,PARAM&gt;</code>          | Modulo of field F by PARAM                                                                                                                           |
| <code>--abs &lt;FIELD&gt;</code>            | Replace signed field F with <code>abs(F)</code>                                                                                                      |
| <code>--sign &lt;FIELD&gt;</code>           | Replace signed field F with sign bit (0/1)                                                                                                           |
| <code>--filter &lt;F,OP,VAL&gt;</code>      | Filter tuples where predicate is false ( <code>=</code> , <code>≠</code> , <code>&lt;</code> , <code>≤</code> , <code>&gt;</code> , <code>≥</code> ) |
| <code>--set &lt;F,PARAM&gt;</code>          | Set field F directly to PARAM (integer, hex, binary, or float)                                                                                       |

## Output Unit & Pattern Options:

|                                                   |                                                                              |
|---------------------------------------------------|------------------------------------------------------------------------------|
| <code>-P, --output-pattern &lt;PATTERN&gt;</code> | Bit pattern to pack output (supports AI floats, multipliers & counter)       |
| <code>--output-unit &lt;BITS&gt;</code>           | Output unit size in bits (default: 8, or auto-inferred from pattern)         |
| <code>--output-raw-unit &lt;BITS&gt;</code>       | Size of repeating output container in bits                                   |
| <code>--output-offset &lt;BITS&gt;</code>         | Bit offset (pregap) of unit inside output container [default: 0]             |
| <code>--output-gap &lt;BITS&gt;</code>            | Bit gap between output containers [default: 0]                               |
| <code>--output-skip-bits &lt;BITS&gt;</code>      | Initial bit offset (skip/prefix) emitted before first container [default: 0] |
| <code>-x, --output-hex</code>                     | Output units as formatted hexadecimal strings                                |
| <code>-b, --output-bits</code>                    | Output units as ASCII bit strings ('0' and '1')                              |
| <code>-i, --output-integers</code>                | Output unsigned integer per unit (one per line)                              |
| <code>-T, --output-tuples</code>                  | Output fields as comma-separated tuples                                      |
| <code>--output-json</code>                        | Output tuples as newline-delimited JSON (NDJSON)                             |
| <code>--json-object</code>                        | Emit JSON as keyed objects instead of positional arrays                      |
| <code>--json-fields &lt;FIELDS&gt;</code>         | Explicit comma-separated field names for JSON/CSV                            |
| <code>--output-csv</code>                         | Output tuples as CSV                                                         |
| <code>--csv-header &lt;HEADER&gt;</code>          | Optional CSV column header row                                               |
| <code>--output-visual</code>                      | Colorized ANSI terminal dump of unaligned fields                             |

## Inspection, Code Generation, Web UI & MCP:

|                                          |                                                                                            |
|------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>--explain-pattern [PATTERN]</code> | Analyze bit layout, byte alignment, and field breakdown                                    |
| <code>--export-c</code>                  | Generate packed C struct (DEPRECATED: use <code>contrib/python/json_to_c.py</code> )       |
| <code>--export-rust</code>               | Generate packed Rust struct (DEPRECATED: use <code>contrib/python/json_to_rust.py</code> ) |
| <code>--probe [FILE]</code>              | Inspect binary entropy, byte classes, periodic strides, and                                |

```

strings
 --probe-units Probe unit stream characteristics and entropy AFTER input
stream processing
 --probe-visual Render visual entropy sparkline and 2D ANSI heatmap
 --probe-keys [SIZE] Scan unit stream for potential maximum-entropy cryptographic
keys [default: 256 bits]
 --probe-field <INDEX> Target specific tuple field index (0-based) for unit probing
after pattern unpacking
 --mcp Launch native JSON-RPC 2.0 Model Context Protocol (MCP)
server

Demuxing & Channel Splitting:
 --demux <FIELD:PATH> Route individual tuple field to a dedicated output file
 --demux-files <PATHS> Comma-separated list of output files for fields 0, 1, 2...
 --input-repeat <COUNT> Repeat input stream N times (0 = infinite)

Merge Options:
 --merge-file <PATH> Interleave stream from secondary file (repeatable for N-way
merge)
 --merge-files <PATHS> Comma- or space-separated list of merge files
 --merge-unit <BITS> Bit width of each merge unit [default: 8]
 --merge-skip-bits <BITS> Initial bit offset in merge file [default: 0]
 --merge-skip-units <UNITS> Skip initial N units in merge file [default: 0]
 --merge-gap <BITS> Bit gap skipped after each merge unit [default: 0]
 --merge-copy-first <BITS> Copy initial header bits from merge file first
 --merge-raw-unit <BITS> Size of repeating merge container in bits
 --merge-offset <BITS> Bit offset (pregap) of unit inside merge container [default:
0]
 --merge-drop-partial-eof Discard incomplete trailing bits at EOF in merge stream
 --merge-no-seek Disable seeking specifically on merge file
 --merge-use-seek Explicitly enable seeking on merge file (default: true)

General:
 --llms, --ai-guide Print high-density agent cheatsheet
 --completions <SHELL> Generate shell completion script (bash, zsh, fish,
powershell, elvish)
 -q, --quiet Silence non-fatal warnings and diagnostic summaries
 -h, --help Print help
 -V, --version Print version

```

## 15. Contributed Real-World Tooling ( [contrib/](#) )

The [contrib/](#) directory provides production-grade reference implementations, standalone shell scripts, Python ctypes tools, and native C programs demonstrating `bdd` :

- **Multimedia & Broadcast Streams** ( [contrib/shell/decode\\_\\*.sh](#) , [contrib/c/decode\\_media.c](#) , [contrib/python/decode\\_media.py](#) ) :
  - MP3 unaligned 32-bit frame header extraction and  $O(1)$  frame seeking.

- MPEG-TS 188-byte container striding and 13-bit PID extraction ( `188B[11:13] → 13` ).
  - RIFF WAV 32-bit stereo audio demuxing and 24-bit to 16-bit PCM downsampling.
  - MP4 ISO-BMFF box traversal and H.264 NAL unit header slicing ( `32U1U2U5U` ).
  - JPEG SOF0 geometry and 4-bit chroma nibble extraction ( `4U4U` ).
- **AI Microscaling Floating-Point Codecs** ( [contrib/shell/decode\\_ai\\_weights.sh](#) , [contrib/c/decode\\_ai\\_weights.c](#) , [contrib/python/decode\\_ai\\_weights.py](#) ):
    - Unpacks sub-byte NVIDIA Blackwell NVFP4 ( `4E` ), OCP FP6 ( `6E` ), FP8 ( `8E / 8M` ), and BF16 into IEEE floats.
    - Slices Hugging Face `.safetensors` headers in  $O(1)$  seek time without Python ML frameworks.
  - **Network Packet Protocol Dissection** ( [contrib/shell/decode\\_network.sh](#) , [contrib/c/decode\\_network.c](#) , [contrib/python/decode\\_network.py](#) ):
    - Slices IPv4, UDP, and TCP headers with discrete single-bit control flags ( `SYN` , `ACK` , `FIN` , `RST` ).
  - **System Memory Forensic Key Discovery** ( [contrib/shell/stream\\_memory\\_keys.sh](#) , [contrib/python/stream\\_memory\\_keys.py](#) ):
    - Streams raw memory dumps ( `/dev/mem` , `/proc/kcore` ) in 256-bit units to locate candidate cryptographic keys.
  - **Linux Kernel Telemetry & Process Lifecycles** ( [contrib/python/bdd\\_netlink\\_proc.py](#) , [contrib/python/bdd\\_ps.py](#) , [contrib/python/bdd\\_top.py](#) ):
    - Full-screen curses top monitor with bit-sliced USS memory metrics from `/proc/[pid]/pagemap` .
    - Real-time kernel process lifecycle monitoring via Netlink connector sockets.

See [contrib/README.md](#) for full documentation and test data files.

---

## 16. Development, Testing, Packaging & Install

---

### Cargo Build Features & Zero-Copy Architecture

`bdd` provides fine-grained Cargo features allowing deployment across standard server platforms, embedded Linux, or minimal WebAssembly targets:

| FEATURE                   | DEFAULT | DESCRIPTION                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>mmap</code>         | Yes     | Enables zero-copy memory-mapped file I/O ( <code>mmap2</code> ) for regular input and merge files. Bypasses <code>read()</code> syscall overhead, provides kernel readahead via <code>MADV_SEQUENTIAL</code> , and enables instant $O(1)$ skipping across container gaps. Seamlessly falls back to buffered streaming for pipes, stdin, FIFOs, and 0-byte files. Can be excluded for targets without OS memory mapping (e.g. WASI, embedded) via <code>--no-default-features</code> . |
| <code>small-floats</code> | Yes     | Hardware and AI microscaling float codecs (FP16, BF16, OCP FP8 E4M3/E5M2, OCP FP6 E3M2, NVIDIA Blackwell FP4 E2M1) provided by the standalone <code>bdd-small-floats</code> subcrate ( <code>crates/bdd-small-floats</code> ). When disabled, reduces binary size and removes all float conversion tables.                                                                                                                                                                            |
| <code>probe</code>        | Yes     | Binary prober, Shannon entropy heatmaps, periodic stride detection, and cryptographic key scanners. Can be excluded via <code>--no-default-features</code> for a minimal, lightweight bitstream slicer.                                                                                                                                                                                                                                                                               |

To build a minimal binary with only standard bit/integer slicing (no probe, no small floats):

```
cargo build --release --no-default-features --features mmap
```

Or to build without memory mapping, small floats, or probing:

```
cargo build --release --no-default-features
```

At runtime, memory mapping can also be explicitly toggled via `--mmap` or disabled via `--no-mmap` .

Test Suite & Verification

```
make test
or: cargo test --all-targets --all-features
```

Run benchmarks:

```
make bench
or: cargo bench
```

Installation from Distribution Packages

Native distribution packages install the full `bdd` toolchain, including `/usr/bin/bdd` , C shared/static libraries ( `libbdd.so` , `libbdd.a` , `bdd.h` , `bdd.pc` ), manpages, and standard format presets at

`/usr/share/bdd/presets.json` (discovered automatically without network access):

- **Fedora / RHEL / CentOS / Rocky ( `.rpm` for DNF):**

```
make rpm
sudo dnf install ./dist/bdd-0.5.3-1.*.rpm
```

- **Debian / Ubuntu / Linux Mint ( `.deb` ):**

```
make deb
sudo apt install ./dist/bdd_0.5.3_amd64.deb
```

- **Rust Cargo Crate:**

```
cargo install --git https://github.com/e-t-u/bdd.git
```

- **Python Module via Pip:**

```
make python
pip install ./dist/bdd-0.5.3-py3-none-any.whl
```

- **C SDK Archive:**

```
make c-lib
```

- **Build All Distribution Packages:**

```
make packages
```

---

## License

GPL-3.0-or-later. Original author: Esa Turtiainen.